

04 — HANA Vector Store & Embeddings (Deep Dive)

Module purpose: turn any document (text, PDF, CSV, Excel, URL) into searchable vectors stored natively in HANA Cloud's REAL_VECTOR column type — the foundation RAG (sheet 05) is built on. Seven files covering the full load → chunk → embed → store → search pipeline.

File 1 — document_loaders.py (multi-format ingestion)

```
def load_from_pdf(file_path: str) -> list[Document]:
    from langchain_community.document_loaders import PyPDFLoader
    loader = PyPDFLoader(file_path)
    docs = loader.load()                                # each PAGE becomes a separate Document
    for i, doc in enumerate(docs):
        doc.metadata.update({"source_type": "pdf", "source_name": Path(file_path).name,
        ↪ "page": i + 1})
    return docs

def load_from_csv(file_path, content_columns=None, metadata_columns=None) -> list[Document]:
    df = pd.read_csv(file_path)
    cols = content_columns or df.columns.tolist()
    docs = []
    for idx, row in df.iterrows():                      # each ROW becomes a
    ↪ separate Document
        content = " | ".join([f"{c}: {row[c]}" for c in cols if c in row])
        meta = {"source_type": "csv", "source_name": Path(file_path).name, "row_index": idx}
        docs.append(Document(page_content=content, metadata=meta))
    return docs

def load_documents(input_source: str, **kwargs) -> list[Document]:
    src = input_source.strip()
    if src.startswith(("http://", "https://")): return load_from_url(src)
    elif src.lower().endswith(".pdf"): return load_from_pdf(src)
    elif src.lower().endswith(".csv"): return load_from_csv(src, **kwargs)
    elif src.lower().endswith(".xlsx", ".xls"): return load_from_excel(src, **kwargs)
    elif src.lower().endswith(".txt"): return load_from_txt_file(src)
    else: return load_from_text(src)                    # fallback: treat the string itself as raw
    ↪ text content
```

Working explanation: every loader returns the same type — list[Document] (LangChain's standard container: page_content + metadata dict) — regardless of source format, so everything downstream (chunker, embedder) works identically no matter what was loaded. The load_documents() auto-detector inspects the string's shape (URL prefix, file extension) to route to the right specific loader — this is a simple but effective “format sniffing” pattern.

File 2 — text_chunker.py

```
from langchain.text_splitter import RecursiveCharacterTextSplitter

def chunk_documents(docs, chunk_size=500, chunk_overlap=50) -> list[Document]:
    splitter = RecursiveCharacterTextSplitter(
        chunk_size=chunk_size,                        # max characters per chunk
        chunk_overlap=chunk_overlap,                  # characters shared between adjacent chunks
        separators=["\n\n", "\n", ". ", " ", ""],    # tries paragraph -> line -> sentence ->
    ↪ word -> char
```

```

)
chunks = splitter.split_documents(docs)
counter = {}
for chunk in chunks:                                # add a sequential index per source
    ↪ document
    src = chunk.metadata.get("source_name", "unknown")
    counter[src] = counter.get(src, 0)
    chunk.metadata["chunk_index"] = counter[src]
    counter[src] += 1
return chunks

```

Working explanation: “recursive” means the splitter tries the FIRST separator (“\n\n” = paragraph breaks) across the whole text; wherever a resulting piece is still too big, it recursively tries the next separator on just that piece, and so on down to raw character splitting as a last resort. This produces chunks that respect natural text boundaries whenever possible, rather than blindly cutting every N characters. `chunk_overlap` exists so a sentence spanning a cut point isn’t fully lost from both neighboring chunks’ context.

File 3 — embedding_setup.py

```

EMBEDDING_MODEL = "text-embedding-ada-002"
VECTOR_DIMENSION = 1536    # must match vector_table.py's REAL_VECTOR(n) – a hard schema
↪ contract

def get_embedding_model():
    deployment_id = os.getenv("EMBEDDING_DEPLOYMENT_ID")
    proxy_client = get_proxy_client("gen-ai-hub")
    return OpenAIEmbeddings(proxy_model_name=EMBEDDING_MODEL, proxy_client=proxy_client,
    ↪ deployment_id=deployment_id)

```

Same pattern covered in sheet 02 — repeated here because vector storage is where dimension mismatches actually surface as runtime errors.

File 4 — vector_table.py (DDL + maintenance)

```

def create_vector_table(conn, table_name="VECTOR_STORE"):
    sql = f"""
    CREATE TABLE IF NOT EXISTS "{table_name}" (
        "ID" NVARCHAR(100) NOT NULL PRIMARY KEY,
        "SOURCE_NAME" NVARCHAR(500), "CHUNK_INDEX" INTEGER,
        "CONTENT" NCLOB, "METADATA" NCLOB,                -- NCLOB = large unicode text
        "EMBEDDING" REAL_VECTOR({VECTOR_DIMENSION})      -- the vector column itself
    )"""
    cursor = conn.cursor(); cursor.execute(sql); conn.commit(); cursor.close()

def clear_vector_table(conn, table_name="VECTOR_STORE", source_name=None):
    cursor = conn.cursor()
    if source_name:
        cursor.execute(f'DELETE FROM "{table_name}" WHERE "SOURCE_NAME" = ?', (source_name,))
    ↪ # scoped delete
    else:
        cursor.execute(f'DELETE FROM "{table_name}"')          # full reset

```

```
count = cursor.rowcount; conn.commit(); cursor.close()
return count
```

```
def get_table_stats(conn, table_name="VECTOR_STORE") -> dict:
    cursor = conn.cursor()
    cursor.execute(f'SELECT COUNT(*), COUNT(DISTINCT "SOURCE_NAME") FROM "{table_name}"')
    row = cursor.fetchone(); cursor.close()
    return {"table": table_name, "total_chunks": row[0], "total_sources": row[1]}
```

Working explanation: IF NOT EXISTS makes table creation idempotent — safe to call on every app startup. `clear_vector_table` supports both a **scoped** delete (re-embedding one changed document without wiping everything) and a **full reset** (`source_name=None`) — knowing which one you're calling matters a lot once you have multiple documents loaded.

File 5 — `embed_and_store.py` (batched write path)

```
def embed_and_store(conn, chunks, embedding_model, table_name="VECTOR_STORE", batch_size=50)
    -> int:
    insert_sql = f"""INSERT INTO "{table_name}"
        ("ID", "SOURCE_TYPE", "SOURCE_NAME", "CHUNK_INDEX", "CONTENT", "METADATA", "EMBEDDING")
        VALUES (?, ?, ?, ?, ?, ?, TO_REAL_VECTOR(?))"""    # TO_REAL_VECTOR parses a string
    into the vector type
    cursor = conn.cursor(); total = 0
    for start in range(0, len(chunks), batch_size):          # process in batches to respect
        embedding API limits
        batch = chunks[start:start + batch_size]
        vectors = embedding_model.embed_documents([c.page_content for c in batch])    # ONE
        API call per batch
        rows = [(str(uuid.uuid4()), c.metadata.get("source_type"),
        c.metadata.get("source_name"),
        c.metadata.get("chunk_index"), c.page_content, json.dumps(c.metadata),
        str(v))
            for c, v in zip(batch, vectors)]
        cursor.executemany(insert_sql, rows)                  # bulk insert, one round trip for
        the whole batch
        conn.commit(); total += len(rows)
    return total
```

Working explanation: vectors are sent to HANA as **strings** (`str(vector) → TO_REAL_VECTOR(?)` parses server-side) — the Python driver has no native vector type, so this string round-trip is required. Batching (`batch_size=50`) exists for two reasons: embedding provider rate limits, and reducing network round trips via `executemany`.

File 6 — `similarity_search.py` (the read path)

```
def similarity_search(conn, query, embedding_model, table_name="VECTOR_STORE", top_k=5,
    source_filter=None) -> list[dict]:
    query_vector = str(embedding_model.embed_query(query))    # embed_query, not
    embed_documents — single vector
    cursor = conn.cursor()
    sql = f"""SELECT TOP {top_k}
    "SOURCE_TYPE", "SOURCE_NAME", "CHUNK_INDEX", "CONTENT", "METADATA",
```

```

        COSINE_SIMILARITY("EMBEDDING", TO_REAL_VECTOR(?)) AS SCORE
        FROM "{table_name}" ORDER BY SCORE DESC""           # HANA computes similarity
↪ server-side, in SQL
    cursor.execute(sql, (query_vector,))
    rows = cursor.fetchall(); columns = [d[0] for d in cursor.description]; cursor.close()
    results = []
    for rank, row in enumerate(rows, start=1):
        rec = dict(zip(columns, row))
        results.append({"rank": rank, "score": round(float(rec["SCORE"]), 4), "content":
↪ rec["CONTENT"],
                        "source_name": rec["SOURCE_NAME"], "metadata":
↪ json.loads(rec["METADATA"] or "{}")})

    return results

```

Working explanation: the entire similarity search happens **inside HANA** via `COSINE_SIMILARITY()` — Python only sends the query vector and reads back already-ranked results (`ORDER BY SCORE DESC, TOP k`). This is far more efficient than pulling every row into Python and computing similarity client-side, and it's the same pattern regardless of table size.

File 7 — `vector_store_agent.py` (wires files 1-6 together)

```

class VectorStoreAgent:
    def __init__(self, conn, table_name="VECTOR_STORE", embedding_model=None):
        self.conn, self.table_name = conn, table_name
        self.embedding_model = embedding_model or get_embedding_model()
        create_vector_table(conn, table_name)           # idempotent — ensures table exists on
↪ startup

    def add(self, input_source, chunk_size=500, chunk_overlap=50, clear_existing=False,
↪ **loader_kwargs) -> int:
        docs = load_documents(input_source, **loader_kwargs)
        if clear_existing and docs:
            clear_vector_table(self.conn, self.table_name,
↪ docs[0].metadata.get("source_name"))
        chunks = chunk_documents(docs, chunk_size, chunk_overlap)
        return embed_and_store(self.conn, chunks, self.embedding_model, self.table_name)

    def search(self, query, top_k=5, source_filter=None) -> list[dict]:
        return similarity_search(self.conn, query, self.embedding_model, self.table_name,
↪ top_k, source_filter)

```

One class, `.add(anything)` and `.search(question)` — the entire 6-file pipeline collapses into two method calls for the caller.

Key Points

- Every loader normalizes to `list[Document]` — a uniform interface regardless of source format.
- Recursive chunking respects natural text boundaries (paragraph → sentence → word) before falling back to raw character cuts.
- Embedding dimension (`VECTOR_DIMENSION`) is a hard contract between the embedding model and the `REAL_VECTOR(n)` table column — must match exactly.
- Vectors travel as **strings** over the wire; `TO_REAL_VECTOR()` parses them server-side on write,

`COSINE_SIMILARITY()` compares them server-side on read.

- Batched embedding (`batch_size=50`) + `executemany` bulk insert — respects API rate limits and minimizes DB round trips.
- `VectorStoreAgent` is the “full pipeline in one class” pattern repeated across every template folder (also seen as `QueryAgent` in sheet 03, `RAGAgent` in sheet 05).

Interview Q&A

Q: Why send the query vector to HANA and compute similarity in SQL, rather than pulling all vectors into Python and comparing there? A: Server-side computation avoids transferring the entire table over the network for every search, scales to large tables, and lets HANA’s engine (and any future vector indexing) optimize the operation — this is the standard “push computation to the data” principle.

Q: What’s the purpose of `chunk_overlap`? A: Prevents a sentence or idea that spans a chunk boundary from being fully lost in both resulting chunks — a small overlap preserves local context at the seams.

Q: How would you re-embed a single updated document without wiping the whole vector store? A: Call `store.add(updated_doc, clear_existing=True)` — internally this calls `clear_vector_table(conn, table, source_name=<that document's name>)` before re-inserting, a scoped delete rather than a full table wipe.

Q: Why are `embed_documents` and `embed_query` two separate calls in this pipeline (not the same function reused)? A: Storage-side (`embed_and_store.py`) always embeds multiple chunks in a batch via `embed_documents`; search-side (`similarity_search.py`) always embeds one query string via `embed_query` — matching the semantic asymmetry some embedding providers use internally, and also because the two call sites naturally have different input shapes (list vs. single string).